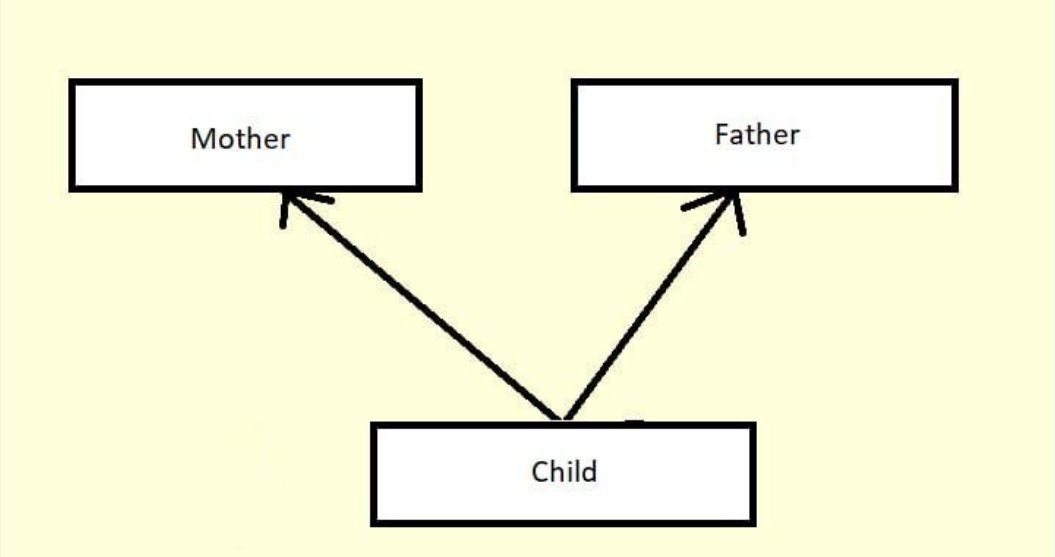
What is Python Multiple Inheritance?

As its name is indicative, multiple inheritance in python is when a class inherits from multiple classes. One example of this would be that a child inherits personality traits from both parents.



Python Multiple Inheritance – Example

Python allows a class to inherit from many parents at once. You write class AmphibiousCar(Car, Boat): to mix driving and sailing skills. This is called multiple inheritance.

Before we proceed to multiple inheritance syntaxes, let's see the python syntax.

Use multiple inheritance with care. It can solve tricky problems—like mixing a Logger class into many tools—but too many parents can tangle code. Simple designs often work best.

Python Multiple Inheritance Syntax

To make a class inherit from multiple python classes, we write the names of these classes inside the parentheses to the derived class while defining it.

We separate these names with commas.

The code for the previous example would be:

```
>>> class Mother:
pass
>>> class Father:
pass
>>> class Child(Mother,Father):
pass
>>> issubclass(Child,Mother) and issubclass(Child,Father)
```

Output

```
True
```

Python MRO (Method Resolution Order)

When we search for an attribute in a class that is involved in python multiple inheritance, an order is followed.

First, it is searched in the current class. If not found, the search moves to parent classes. This is left-to-right, depth-first.

So, in the above class, the search order will be – Child, Mother, Father, Object.

This order is called linearization of class Child, and the set of rules applied are called MRO (Method Resolution Order).

To get the MRO of a class, you can use either the __mro__ attribute or the mro() method.

```
>>> Child.__mro__
```

Output

```
(<class '__main__.Child'>, <class '__main__.Mother'>, <class '__main__.Father'>, <class 'object'>)
```

The __mro__ attribute returns a tuple, but the mro() method returns a python list.

```
>>> Child.mro()
```

Output

```
[<class '__main__.Child'>, <class '__main__.Mother'>, <class '__main__.Father'>, <class 'object'>]
```

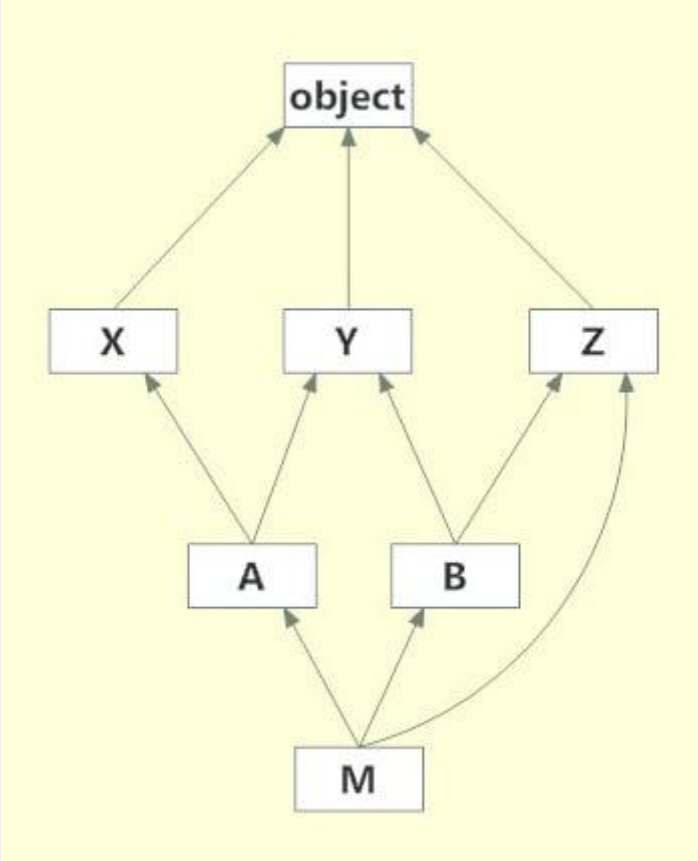
To take a more complex example that also demonstrates depth-first search, we take 6 classes.

```
>>> class X:pass
>>> class Y: pass
>>> class Z:pass
>>> class A(X,Y):pass
>>> class B(Y,Z):pass
>>> class M(B,A,Z):pass
>>> M.mro()
```

Output

```
[<class '__main__.M'>, <class '__main__.B'>, <class '__main__.A'>, <class '__main__.X'>, <class '__main__.Y'>, <class '__main__.Z'>, <class 'object'>]
```

We can represent this with the following diagram.



Python Multiple Inheritance – Method Resolution order (MRO)

First, the interpreter scans M. Then, it scans B, and then A-B first because of the order of arguments at the time of inheritance.

It scans Z later, after X and Y. The order is- X, then Y, then Z. This is because due to depth-first search, X comes first to A.

Finally, it scans the class object. Hence, the order.

Complications in Python Multiple Inheritance

What happens when the classes we inherit from all have a common attribute? Whose value does the child class take, then?

Let's take three classes A, B, and C.

```
>>> class A:
id=1
>>> class B:
id=2
>>> class C:
id=3
>>> class M(A,C,B):
pass
>>> M.id
```

Output

```
1
```

```
>>> M.id
```

Output

```
1
```

```
>>> class M(C,B,A):
pass
>>> M.id
```

Output

```
3
```

Like we see here, the class named first in the inheritance passes its value to the child class for the common attribute.

Earlier, it was A, so, M had id=1. Then, when we changed it to C, M got id=3. This is the same with Python methods/functions of the class.

```
>>> class A:
def sayhi():
print("A")
>>> class B:
def sayhi():
print("B")
>>> class M(A,B):
pass
>>> M.sayhi()
```

Output

```
A
```

So, this was all about Python Multiple Inheritance Tutorial. Hope you like our explanation on Python MRO.

Resources:

https://data-flair.training/blogs/python-multiple-inheritance/